

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Friday 11th September, 2026

Chapter 1

Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on the experimental validation of the method and its ability to consistently identify meaningful conceptual structures. The remainder of this chapter is organized as follows.

- Section 1.1 describes the software implementation of the CAR pipeline.
- Section 1.2 details the experimental configuration, including the environments, the reinforcement learning setup, and the training of the surrogate policies.
- Section 1.3 presents and analyses the results obtained by applying CAR and its metrics.
- Section ?? summarizes the findings and discusses their implications.

1.1 Software Implementation

Deep Learning (DL) and Reinforcement Learning (RL) are inherently empirical fields, in which the effectiveness of a method is assessed through experimentation. For this reason, particular attention is given to the implementation of CAR. It relies on established libraries that provide efficient and widely used implementations of its main components. The implementation is made publicly available as open-source to support the objective and reproducible replication of the reported results.¹

CAR is organized as a pipeline of five stages, shown in Figure 1.1. Each stage is an independent executable, and stages communicate only through artifacts written to disk. This decoupling allows any stage to be rerun or inspected independently. The parameters of each experiment are gathered in a single configuration object, described in Section 1.2. Every result can therefore be traced back to the settings that produced it.

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space

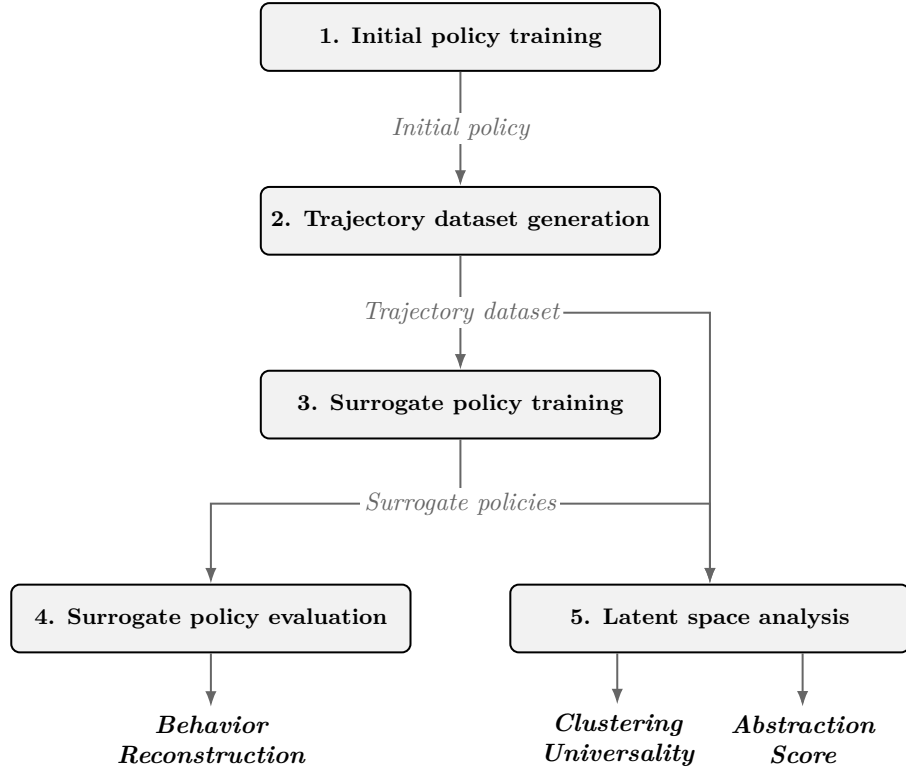


Figure 1.1: The CAR pipeline.

1.1.1 Initial policy training

The first stage trains the initial policy π using RLlib [Moritz et al., 2018, Liang et al., 2018]. RLlib provides a highly scalable implementation of RL algorithms and supports a wide range of state-of-the-art methods. It supports distributed training, allowing the computational workload to be spread across multiple processes and machines.

The use of RLlib allows CAR to be applied to any environment implemented using the Gymnasium interface [Towers et al., 2024]. This provides a common interface for environments with different observation and action spaces. This flexibility allows CAR to be applied to a wide range of environments.

By default, the RL algorithm used for the initial policy is Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Other algorithms supported by RLlib, such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3, can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [Paszke et al., 2019]. PyTorch provides efficient GPU computation and supports a wide range of neural network architectures. This provides a standardized implementation that is compatible with the broader DL ecosystem.

The output of this stage is a checkpoint containing the trained policy π .

51 1.1.2 Trajectory dataset generation

52 The second stage builds the dataset used to train the surrogate policies. The
 53 distributed execution capabilities of RLlib are used to collect trajectories in
 54 parallel. Multiple environment runners execute the original policy simultaneously,
 55 distributing the computational workload across the available resources.

56 The original policy is run in the environment, and the interactions between the
 57 agent and the environment are recorded. For each step, the dataset stores the ob-
 58 servation together with the parameters of the action distribution produced by the
 59 policy. Trajectories are stored as Parquet files [The Apache Software Foundation, 2016].
 60 Parquet is a compressed columnar format widely used for large-scale data pro-
 61 cessing. Its columnar structure allows data to be read efficiently and supports
 62 concurrent access from multiple processes. This is particularly useful here be-
 63 cause the datasets contain on the order of one million observation-action pairs
 64 and cannot be loaded entirely into memory.

65 1.1.3 Surrogate policy training

66 The third stage trains the surrogate policies using behavior cloning. This stage
 67 therefore converts the original RL problem into a supervised learning problem.
 68 The surrogate policies are trained to reproduce the action distributions generated
 69 by π from the collected observations.

70 The training procedure is implemented using PyTorch Lightning [Falcon and The PyTorch Lightning team,
 71 PyTorch Lightning provides a structured interface for supervised training while
 72 remaining compatible with the PyTorch ecosystem. It handles common training
 73 operations such as data loading, optimization, checkpointing, and GPU execu-
 74 tion, while allowing the underlying neural network implementation to remain
 75 fully controlled.

76 Once training is complete, the stage produces a set of n trained surrogate
 77 policies.

78 1.1.4 Surrogate policy evaluation

79 The fourth stage measures how faithfully each surrogate policy reproduces the
 80 behavior of π . Each surrogate is wrapped so that it can be used as an RLlib
 81 policy and is then evaluated in the original environment. Its mean cumulative
 82 reward over multiple episodes is compared with that of π . Evaluation episodes
 83 are collected using parallel environment runners.

84 This stage provides the value of the *Behavior Reconstruction*. It therefore
 85 verifies whether the surrogate policies adequately reproduce the behavior of the
 86 original policy before their latent representations are analyzed. If a surrogate
 87 does not reproduce the original behavior sufficiently well, its training can be
 88 repeated before proceeding to the next stage.

89 1.1.5 Latent space analysis

90 The fifth stage performs the main analysis of CAR. The latent representations
 91 produced by the surrogate policies form the point clouds that are analyzed using
 92 clustering algorithms.

93 The clustering step relies on the cuML library [RAPIDS Development Team, 2023],
 94 which provides GPU-accelerated implementations of several clustering algorithms.

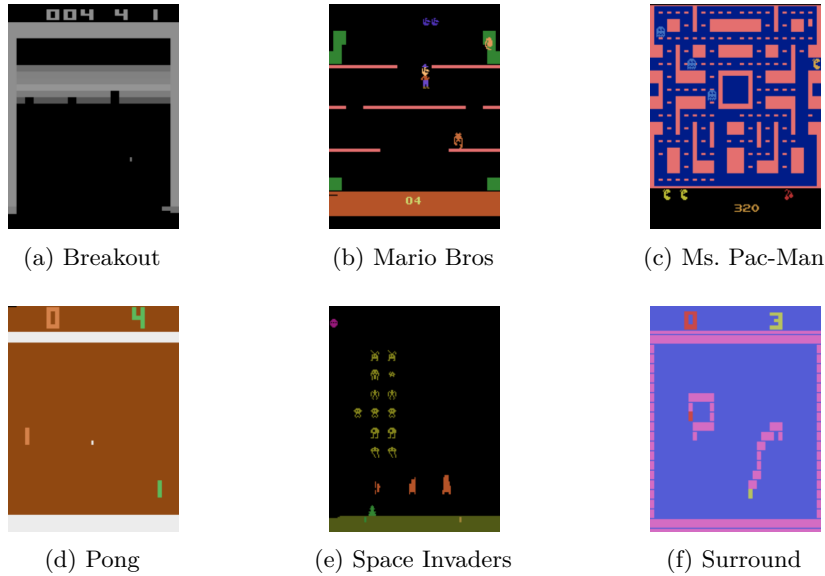


Figure 1.2: The six Atari environments used to evaluate CAR, shown as rendered game frames. The agent does not receive these frames directly: each observation is a stack of four consecutive frames, resized to 64×64 pixels and converted to grayscale, except for Ms. Pac-Man, which is kept in color.

This significantly reduces the computational cost of clustering and makes it possible to analyze point clouds containing tens of thousands of representations.

The resulting partitions are used to compute the *Clustering Universality* and the *Abstraction Score*. The outputs of this stage are the resulting metric values and the material required for the qualitative analysis presented in the following sections.

1.2 Experimental Configuration

This section describes the setup shared by all the experiments reported in this chapter: the environments, the neural architecture of the policies, and the main parameters of each pipeline stage. For every environment, a single initial policy is trained and then imitated by $n = 3$ surrogate policies.

1.2.1 Environments

All experiments use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment and configured following common practice in reinforcement learning research [Machado et al., 2017]. In every environment the agent receives visual observations and selects actions from a discrete set. The six environments are shown in Figure 1.2.

Breakout. The agent moves a paddle at the bottom of the screen (Figure 1.2a) to bounce a ball into a wall of bricks, each broken brick scoring a point, and the

114 ball speeding up as the wall is cleared. The observation is dominated by the
 115 layout of the remaining bricks, which changes gradually over a game.

116 **Mario Bros.** A single-screen platform game (Figure 1.2b). The player clears
 117 each phase by hitting enemies from below to flip them, then kicking them off
 118 the platforms before they recover. A central POW block flips every enemy at
 119 once but can only be used a few times per phase. Deciding when to spend the
 120 POW block is a clear strategic choice, which makes this environment useful for
 121 testing whether CAR recovers strategy-level distinctions.

122 **Ms. Pac-Man.** The player moves through a maze (Figure 1.2c), eating pellets
 123 while avoiding four ghosts, with power pellets briefly making the ghosts edible.
 124 The maze structure and the many possible local situations, such as a nearby
 125 ghost, an intersection, or an active power pellet, produce a large number of
 126 distinguishable states. This makes Ms. Pac-Man well suited to examining CAR
 127 at fine cluster granularities. It is also the only environment kept in color.

128 **Pong.** A minimal two-player game (Figure 1.2d) in which the agent controls
 129 one paddle and must send the ball past the opponent. The observation contains
 130 almost nothing beyond the two paddles, the ball, and the score. Pong therefore
 131 acts as a lower bound on how much structure CAR can be expected to recover.

132 **Space Invaders.** The agent moves a cannon along the bottom of the screen
 133 (Figure 1.2e), firing at descending rows of aliens while dodging their shots and
 134 using destructible shields, with a bonus saucer occasionally crossing the top. A
 135 game goes through visually distinct phases, from the full alien formation to a
 136 few fast aliens near the ground, which makes this environment convenient for
 137 illustrating the clusters found by CAR.

138 **Surround.** A two-player game on a grid (Figure 1.2f). Each player leaves a
 139 solid trail behind a moving cursor, and the goal is to force the opponent to crash
 140 into a trail or a wall. Every move permanently blocks a cell, so the state of a
 141 game is essentially the pattern of trails drawn so far, which grows continuously
 142 over an episode.

143 1.2.2 Neural architecture

144 The initial policy and the surrogate policies are convolutional networks that share
 145 the same feature-extraction trunk. The trunk is a sequence of four convolutional
 146 layers, with 16, 32, 64, and 128 channels, 4×4 kernels, and a stride of 2, each
 147 followed by layer normalization and a LeakyReLU activation. Its input is the
 148 observation, a stack of four consecutive 64×64 frames.

149 The initial policy follows an actor-critic design: the trunk feeds two separate
 150 multilayer perceptrons, one producing the parameters of the action distribution
 151 and one producing the value estimate. The surrogate policy has the encoder-
 152 decoder structure introduced in Section ??: its encoder f_{enc} is a trunk with the
 153 same architecture, and its decoder is a single multilayer perceptron producing
 154 the parameters of the action distribution. The surrogate has no value head, since
 155 it is trained only to reproduce the action distribution of the initial policy.

Observations	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Color	grayscale (color for Ms. Pac-Man)
Initial policy training (PPO)	
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training (behavior cloning)	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Optimizer	Adam
Learning rate	1×10^{-4}
Batch size	2048
Loss	MSE between action-distribution parameters
Train / validation split	80 / 20
Latent space analysis	
Observations per surrogate policy	50 000
Clustering algorithm	k -means, Euclidean distance, 10 restarts
Number of clusters K	2 to 30
Points for the 2D projection	10 000

Table 1.1: Parameters common to all experiments, grouped by pipeline stage.

156 The latent space analyzed by CAR is the representation produced by the
157 last convolutional layer of the surrogate encoder.

158 1.2.3 Parameters

159 Table 1.1 lists the parameters common to all experiments, grouped by pipeline
160 stage. Parameters that vary across environments, such as the reward scaling and
161 the exact number of training epochs, are given in the released configuration files.

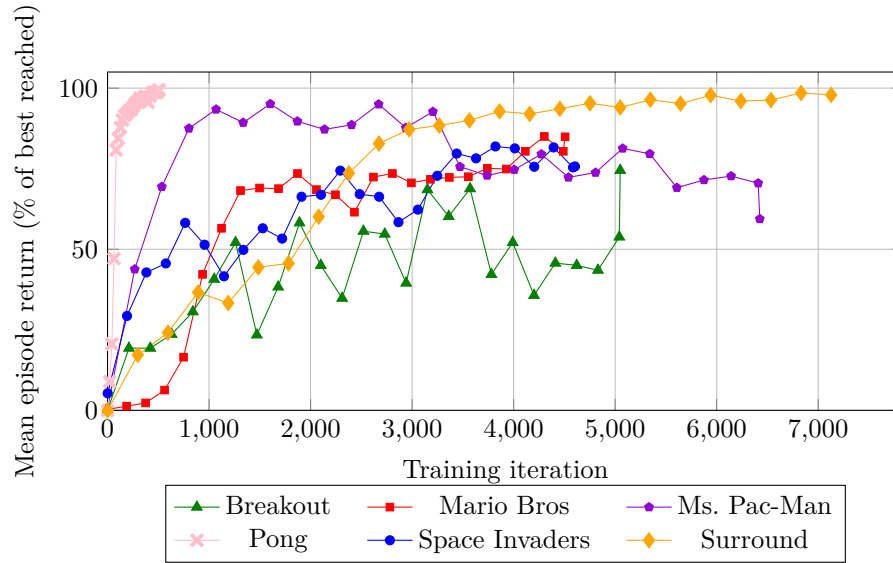


Figure 1.3: Training of the initial policies with PPO, one curve per environment.

1.3 Results

This section reports the results of applying CAR to the six environments described in Section 1.2. Sections 1.3.1 and 1.3.2 first check that the two learning stages of the pipeline behave as expected: that the initial policies reach a competent level of play, and that the surrogate policies faithfully imitate them. Section 1.3.3 then presents the analysis of the latent representations and the values of the three evaluation metrics.

1.3.1 Initial policy training

Figure 1.3 shows how the initial policies improve during PPO training. For each environment, the mean episode return is normalized so that 0% and 100% correspond to the lowest and highest mean return observed during that run. In every environment the return increases and then reaches a plateau, at which point training is stopped. The number of iterations required to reach this plateau varies from a few hundred, for Pong, to several thousand, for Surround and Ms. Pac-Man. For Ms. Pac-Man the return decreases after its peak; since the best checkpoint is retained rather than the last one, this does not affect the policy that is subsequently analyzed.

To confirm that the trained policies play competently, their scores are compared with the reference scores reported by [Machado et al., 2017]. Each initial policy reaches a score of the same order as these references, so it constitutes a reasonable target for imitation.

1.3.2 Surrogate policy training

Figure 1.4 shows, for each environment, the training action loss of the three surrogate policies: the mean squared error between the action-distribution

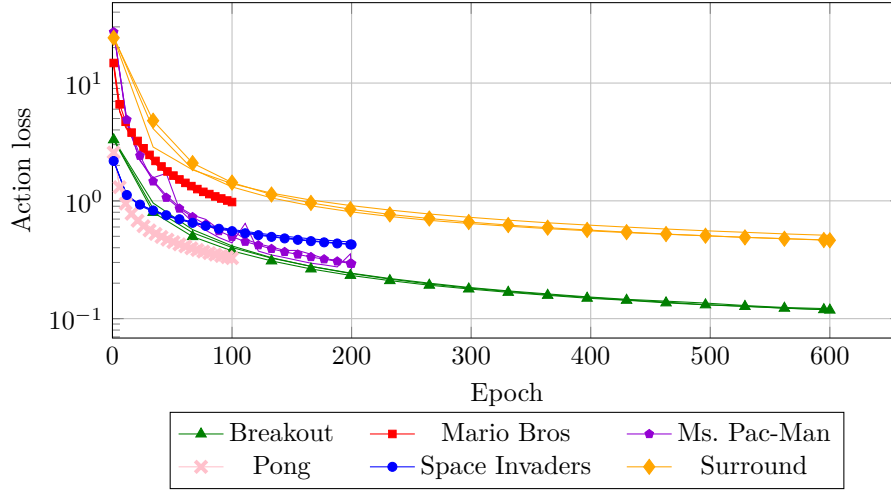


Figure 1.4: Training action loss of the surrogate policies, on a logarithmic scale, three curves per environment.

parameters of the surrogate and those of the initial policy. In every case the loss decreases smoothly, and the three independently trained policies follow almost identical trajectories, converging to the same value; the three curves of each environment largely overlap. This consistency matters for CAR: the method compares the latent representations of independently trained policies, and this comparison is only meaningful if these policies implement the same decision function. The behavioral fidelity of the surrogate policies is quantified by the *Behavior Reconstruction* in the next section.

1.3.3 Latent space analysis

We first evaluate the ability of the surrogate policies to replicate the behavior of the initial policies (*Behavior Reconstruction*, Table 1.2). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for Breakout to 1.00 for Ms. Pac-Man and Surround. They therefore reproduce the decisions of the initial policies closely enough for their latent representations to be analyzed in place of those of the initial policies.

Environment	<i>Behavior Reconstruction</i>	<i>Abstraction Score</i>	<i>Clustering Universality</i>
Breakout	0.83 ± 0.03	0.82 ± 0.07	0.48 ± 0.11
Mario Bros	0.99 ± 0.03	0.71 ± 0.14	0.55 ± 0.10
Ms. Pac-Man	1.00 ± 0.03	0.43 ± 0.09	0.65 ± 0.08
Pong	0.95 ± 0.01	0.99 ± 0.00	0.49 ± 0.05
Space Invaders	0.97 ± 0.07	0.69 ± 0.06	0.67 ± 0.07
Surround	1.00 ± 0.01	0.55 ± 0.07	0.61 ± 0.09

Table 1.2: Evaluation of *Behavior Reconstruction*, *Abstraction Score* and *Clustering Universality* across the six Atari environments.

For the remainder of the analysis, a random subset of 50,000 observations is embedded into the Latent Space (LS) of each surrogate policy, using the representation extracted directly after the last convolutional layer, and the resulting point cloud $\mathcal{E}$ is partitioned with k -means clustering [MacQueen, 1967].

Figure 1.5 reports the evolution of *Clustering Universality* with respect to the number of clusters. After the first few values of K , every curve settles onto a plateau whose level depends on the environment, from about 0.45–0.50 for Breakout and Pong to about 0.65 for Ms. Pac-Man and Space Invaders. Since *Clustering Universality* is computed from the Adjusted Rand Index (ARI), and an ARI of 0.5 already corresponds to substantial agreement [Amezquita et al., 2020], these values indicate that independently trained surrogate policies recover a similar cluster structure in every environment, and that this structure is stable with respect to the number of clusters. The strength of this agreement varies across environments, but remains clearly above chance everywhere. The absence of a single optimal number of clusters suggests that K should be chosen according to the level of granularity desired for the analysis.

The *Abstraction Score*, in contrast, varies markedly across environments. It is close to 1 for Pong (0.99) and high for Breakout, Mario Bros and Space Invaders (0.69 to 0.82): in these environments the latent clusters are largely independent of a direct clustering of the observations or of the action distributions. It is lower for Surround (0.55) and Ms. Pac-Man (0.43). In these two environments the position of the agent on the screen largely determines both the raw observation and the decision to take, so a clustering of the LS naturally coincides with a clustering of the observations, and the *Abstraction Score* cannot separate the two; a low score does not by itself imply that the representation is not abstract.

With fewer clusters, CAR highlights broad strategic patterns. For instance, in the Mario Bros environment, the two main groups correspond to whether the POW block is present (highlighted in red in Figure 1.7A) or not. This is particularly interesting, as the use of the POW block is a key determinant of strategy in this environment [AtariAge, 2024].

With a larger number of clusters, the analysis captures more fine-grained tactical behaviors, such as “Attacking the left” and “Navigating through the level” (see Figure 1.7B).

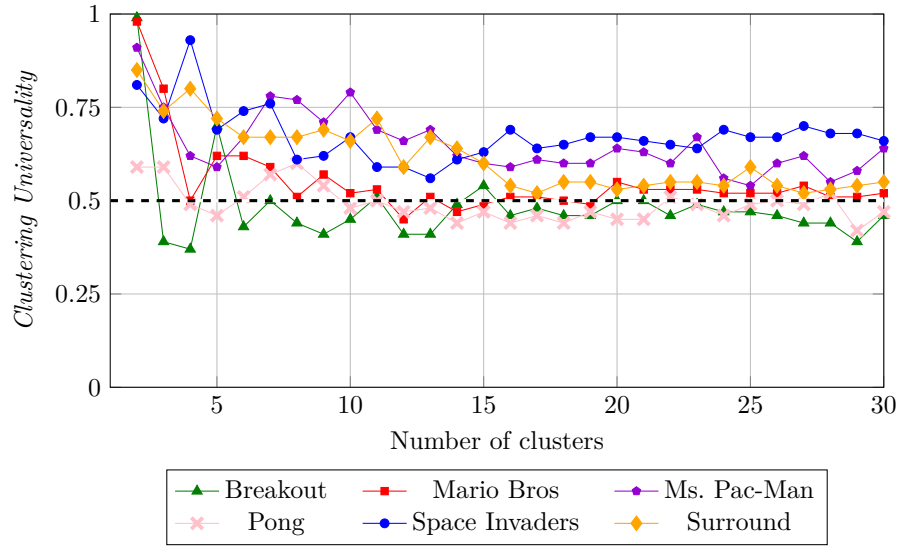


Figure 1.5: Evolution of *Clustering Universality* with respect to the number of clusters, for each of the six Atari environments.

Both perspectives are valid, depending on whether the goal is to study strategic or tactical behaviors. Similar interpretable structures can be observed across the evaluated environments.

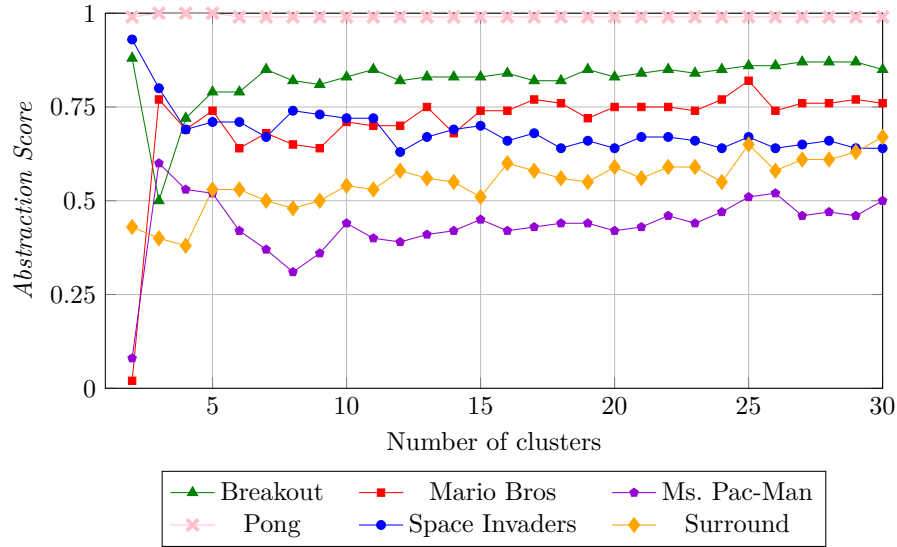
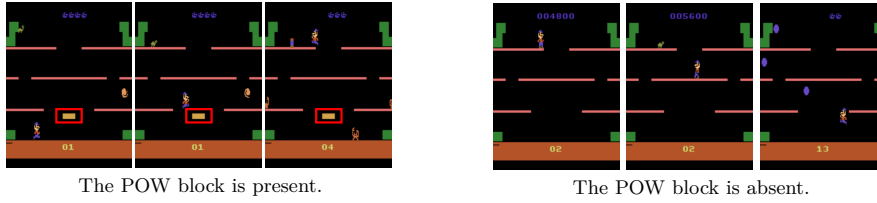


Figure 1.6: Evolution of *Abstraction Score* with respect to the number of clusters, for each of the six Atari environments.

(A) The case with 2 clusters.



(B) The case with 5 clusters.

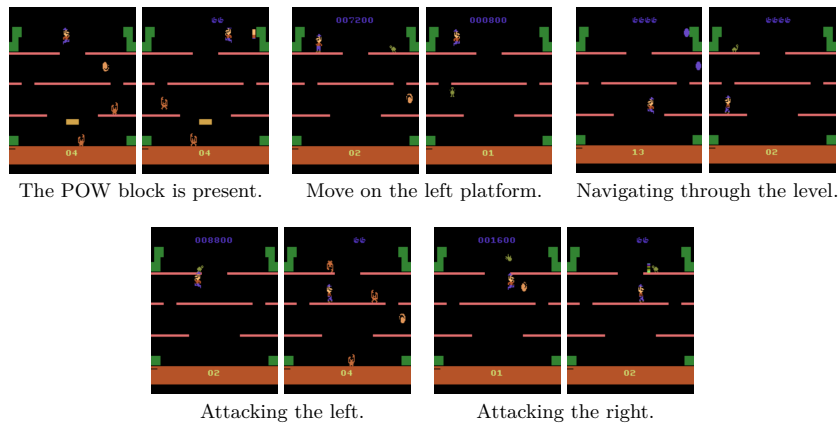


Figure 1.7: Effect of the number of clusters on the groupings identified by CAR in the Mario Bros environment.

237 Acronyms

238 **APPO** Asynchronous Proximal Policy Optimization 2

239 **ARI** Adjusted Rand Index 9

240 **CAR** Clustering Agent Representation 1–9, 11

241 **DL** Deep Learning 1, 2

242 **DQN** Deep Q-Networks 2

243 **LS** Latent Space 9

244 **PPO** Proximal Policy Optimization 2, 7

245 **RL** Reinforcement Learning 1–3

Bibliography

- [Amezquita et al., 2020] Amezquita, R. A., Lun, A. T., Becht, E., Carey, V. J., Carpp, L. N., Geistlinger, L., Marini, F., Rue-Albrecht, K., Risso, D., Soneson, C., et al. (2020). Orchestrating single-cell analysis with Bioconductor, volume 17. Nature Publishing Group US New York. **Book**.
- [AtariAge, 2024] AtariAge (2024). Mario bros. atari 2600. **Web**.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article**.
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon and The PyTorch Lightning team, 2019] Falcon, W. and The PyTorch Lightning team (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web**.
- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article**.
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018).

- 280 Ray: A distributed framework for emerging AI applications. In 13th USENIX
281 Symposium on Operating Systems Design and Implementation (OSDI), pages
282 561–577.
- 283 [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J.,
284 Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison,
285 A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy,
286 S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An
287 imperative style, high-performance deep learning library. In Advances in
288 Neural Information Processing Systems (NeurIPS).
- 289 [RAPIDS Development Team, 2023] RAPIDS Development Team (2023).
290 RAPIDS cuML: GPU-accelerated machine learning. [https://github.com/](https://github.com/rapidsai/cuml)
291 [rapidsai/cuml](https://github.com/rapidsai/cuml).
- 292 [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and
293 Klimov, O. (2017). Proximal policy optimization algorithms.
- 294 [The Apache Software Foundation, 2016] The Apache Software Foundation
295 (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- 296 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
297 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
298 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
299 environments.